

Лабораторная работа №1

Многомерная оптимизация сложной функции генетическим алгоритмом

Федоровская Е.С., Б24-517

1 Постановка задачи и индивидуальный вариант

Требуется найти приближённое глобально оптимальное решение задачи минимизации многомерной функции с помощью генетического алгоритма.

Для определения индивидуального варианта используются:

$$N = 25, \quad Q = 1, \quad Y = 26.$$

Номер варианта:

$$V = 1 + ((17N + 7Q + Y) \bmod 20).$$

После подстановки значений:

$$V = 1 + ((17 \cdot 25 + 7 \cdot 1 + 26) \bmod 20) = 19.$$

Размерность задачи:

$$d = 5 + (V \bmod 6) = 6.$$

Для варианта 19 используется функция HarryCat:

$$f(x) = \left| \sum_{i=1}^d x_i^2 - d \right|^{1/4} + \frac{0.5 \sum_{i=1}^d x_i^2 + \sum_{i=1}^d x_i}{d} + 0.5.$$

Область поиска:

$$-20 \leq x_i \leq 20.$$

Теоретическим ориентиром служит точка

$$x^* = (-1, -1, -1, -1, -1, -1),$$

для которой

$$f(x^*) = 0.$$

2 Пространство решений и представление особи

Решение представляется вещественным вектором

$$x = (x_1, x_2, \dots, x_6), \quad x_i \in [-20, 20].$$

Генотипом является сам вещественный вектор длины 6. Фенотип совпадает с генотипом, поэтому отдельная процедура декодирования не требуется.

Один ген соответствует одной координате вектора. Размер популяции равен 150 особям.

Для каждой особи вычисляется значение функции HarryCat. Поскольку решается задача минимизации, чем меньше значение целевой функции, тем лучше считается решение.

3 Описание генетического алгоритма

3.1 Инициализация

Начальная популяция из 150 особей генерируется равномерно в допустимой области:

$$x_i \sim U(-20, 20).$$

3.2 Оценивание

Для каждой особи вычисляется значение функции `HappyCat`. Полученное значение используется как оценка качества решения.

3.3 Турнирная селекция

Для выбора каждого родителя случайно выбираются три особи из текущей популяции. Победителем турнира становится особь с минимальным значением целевой функции.

Размер турнира:

$$k = 3.$$

Такой способ селекции даёт преимущество более качественным решениям, но не ограничивает выбор родителей только одной лучшей особью и позволяет сохранять разнообразие популяции.

3.4 Арифметический crossover

Для двух родителей p_1 и p_2 генерируется вектор

$$\alpha \in [0, 1]^6.$$

Потомки вычисляются по формулам:

$$c_1 = \alpha p_1 + (1 - \alpha)p_2,$$

$$c_2 = (1 - \alpha)p_1 + \alpha p_2.$$

Вероятность применения crossover равна

$$p_c = 0.90.$$

Если crossover не применяется, в потомство переходят копии родителей без смешивания.

3.5 Гауссовская мутация

Каждый ген потомка мутирует независимо с заданной вероятностью. Для основной конфигурации:

$$p_{mut} = 0.30.$$

При мутации к координате добавляется случайный шум из нормального распределения:

$$N(0, \sigma^2).$$

В работе реализован адаптивный масштаб гауссовской мутации. Значение σ уменьшается от

$$\sigma_{start} = 1.0$$

в начале работы алгоритма до

$$\sigma_{end} = 0.001$$

в конце.

Большое значение σ в начале позволяет шире исследовать пространство поиска, а малое значение в конце — точнее улучшать уже найденные решения.

3.6 Обработка ограничений

После мутации отдельные координаты могут выйти за пределы допустимой области. Для восстановления допустимости используется ограничение:

$$x_i = \min(20, \max(-20, x_i)).$$

В программе это реализовано с помощью `np.clip`.

3.7 Элитизм

Две лучшие особи текущего поколения переносятся в следующее поколение без изменений.

Это позволяет не потерять уже найденные хорошие решения из-за случайного crossover или мутации.

3.8 Критерий остановки

Алгоритм работает до исчерпания фиксированного бюджета в 30000 вычислений целевой функции.

4 Общая схема работы алгоритма

На рисунке показана общая последовательность действий генетического алгоритма.

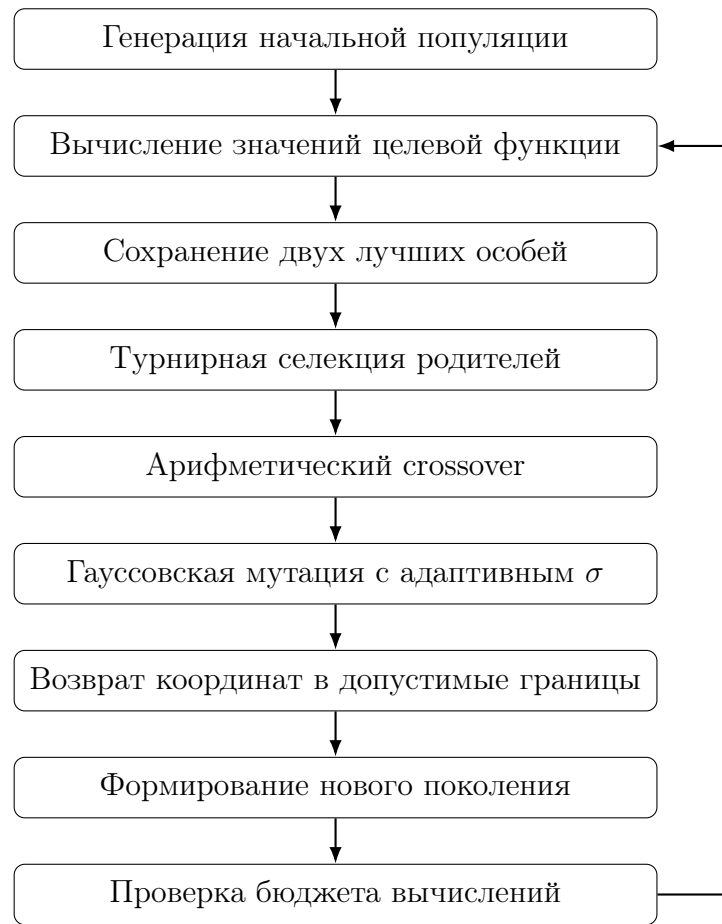


Рис. 1: Общая схема генетического алгоритма

5 Обоснование выбора операторов

В задаче используется вещественное представление, поэтому арифметический crossover естественно формирует новые вещественные решения из координат родителей и не требует дополнительного декодирования.

Турнирная селекция выбрана как простой способ дать преимущество более качественным особям без необходимости масштабировать значения целевой функции.

Гауссовская мутация подходит для непрерывных параметров, поскольку изменяет координаты на вещественные величины. Адаптивное уменьшение σ позволяет сочетать более широкий поиск на ранних этапах и более точную настройку на поздних.

Элитизм сохраняет лучшие уже найденные решения. Операция `np.clip` обеспечивает выполнение ограничений $-20 \leq x_i \leq 20$ после мутации.

6 План экспериментов

Для каждой конфигурации проведено 20 независимых запусков с различными фиксированными значениями `seed`.

Сравнивались три варианта:

1. генетический алгоритм с $p_{mut} = 0.15$;
2. генетический алгоритм с $p_{mut} = 0.30$;
3. равномерный случайный поиск.

При сравнении двух конфигураций генетического алгоритма изменяется только один фактор — вероятность мутации. Остальные параметры остаются одинаковыми.

Для случайного поиска используется тот же бюджет в 30000 вычислений функции, поэтому методы сравниваются при одинаковом числе обращений к целевой функции.

7 Параметры алгоритма

Таблица 1: Параметры генетического алгоритма

Параметр	Значение
Размерность	6
Нижняя граница	-20
Верхняя граница	20
Размер популяции	150
Бюджет вычислений	30000
Вероятность crossover	0.90
Основная вероятность мутации	0.30
Размер турнира	3
Количество элитных особей	2
Начальное значение σ	1.0
Конечное значение σ	0.001
Число независимых запусков	20

Полная таблица параметров сохраняется в файле `results/parameters.csv`.

8 Результаты экспериментов

Итоговая статистика по 20 независимым запускам:

Таблица 2: Результаты серии запусков

Метод	Лучшее	Среднее	Медиана	Std	Худшее
GA, $p_{mut} = 0.15$	0.017928	0.055756	0.053582	0.019079	0.090159
GA, $p_{mut} = 0.30$	0.023792	0.037611	0.034889	0.009478	0.055059
Random search	2.132033	4.275602	4.277174	1.184828	6.056624

Поскольку решается задача минимизации, меньшие значения соответствуют лучшим результатам.

8.1 Сходимость генетического алгоритма

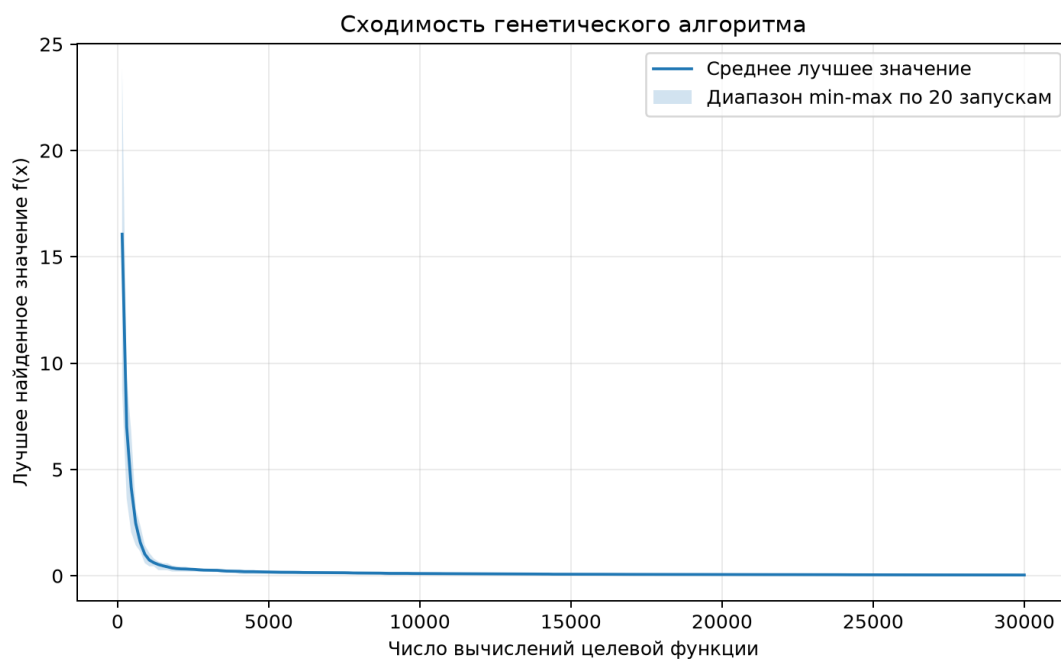


Рис. 2: Сходимость генетического алгоритма

По мере увеличения числа вычислений целевой функции лучшее найденное значение уменьшается. Это означает, что в процессе эволюции алгоритм находит всё более качественные решения.

Разброс между независимыми запусками отражает влияние случайности на результат работы генетического алгоритма.

8.2 Влияние вероятности мутации

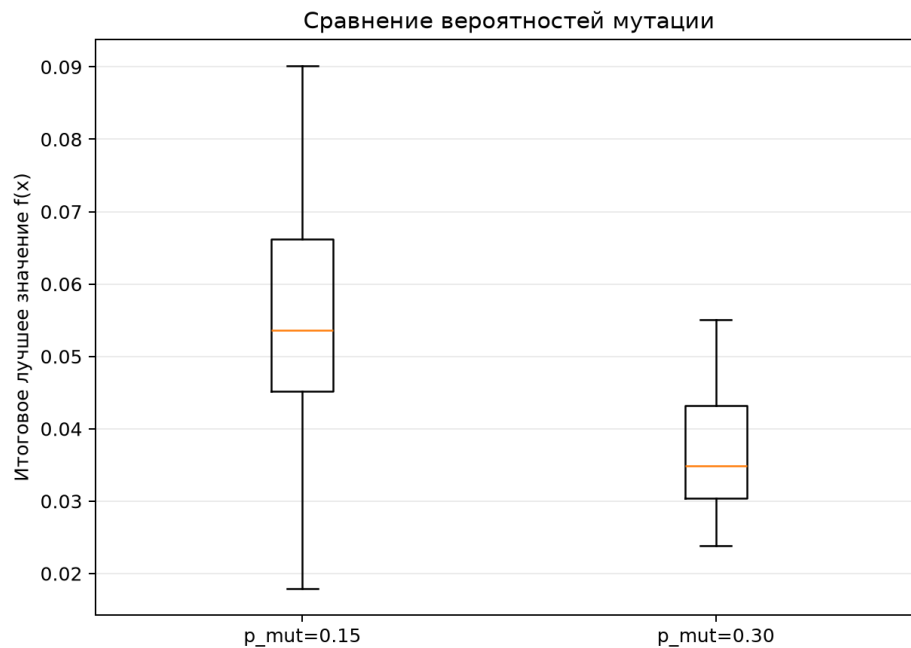


Рис. 3: Сравнение вероятностей мутации

Конфигурация с $p_{mut} = 0.30$ имеет меньшее среднее значение, медиану и стандартное отклонение по сравнению с $p_{mut} = 0.15$.

Это означает, что в проведённой серии запусков вероятность мутации 0.30 дала более устойчивые результаты.

При этом абсолютный лучший единичный результат был получен при $p_{mut} = 0.15$. Поэтому оценивать качество стохастического алгоритма только по одному удачному запуску некорректно.

8.3 Сравнение со случайным поиском



Рис. 4: Сравнение генетического алгоритма со случайным поиском

При одинаковом бюджете в 30000 вычислений функции генетический алгоритм показывает значительно меньшие итоговые значения целевой функции, чем равномерный случайный поиск.

Среднее итоговое значение для основной конфигурации генетического алгоритма составляет 0.037611, тогда как для случайного поиска — 4.275602.

9 Почему случайный поиск менее эффективен

Область поиска является шестимерной:

$$[-20, 20]^6.$$

При равномерном случайном поиске каждая следующая точка генерируется независимо от ранее найденных решений. Даже если была обнаружена перспективная область пространства, эта информация не используется при последующих шагах.

Генетический алгоритм использует результаты предыдущих поколений: более качественные особи получают преимущество при выборе родителей, их координаты комбинируются при crossover, а мутация обеспечивает дополнительное исследование пространства поиска.

Поэтому поиск постепенно концентрируется в более перспективных областях, тогда как случайный поиск продолжает равномерно выбирать точки по всей области.

10 Оценка качества результата

Теоретическим ориентиром для данной функции является значение

$$f(x^*) = 0.$$

В 20 независимых запусках генетический алгоритм с $p_{mut} = 0.30$ получил среднее итоговое значение 0.037611, а лучший результат для исследованных конфигураций составил 0.017928.

Эти значения близки к теоретическому ориентиру и значительно лучше результатов равномерного случайного поиска при том же бюджете вычислений.

Поэтому полученный результат можно считать хорошим для поставленной задачи.

11 Вывод

В работе реализован генетический алгоритм для минимизации функции HarryCat варианта 19.

Использовались вещественное представление особи, турнирная селекция, арифметический crossover, гауссовская мутация с адаптивным масштабом, обработка ограничений и элитизм.

По результатам 20 независимых запусков обе конфигурации генетического алгоритма значительно превзошли равномерный случайный поиск при одинаковом бюджете вычислений.

Конфигурация с $p_{mut} = 0.30$ показала меньшее среднее значение, медиану и стандартное отклонение, то есть более устойчивое качество в данной серии запусков. При этом лучший единичный результат был получен при $p_{mut} = 0.15$, что подтверждает необходимость оценивать стохастический алгоритм по серии независимых запусков, а не по одному результату.